REACT WEB

Budget Tracker — подробный разбор

Пошаговое учебное руководство
от условия до проверки и сдачи

Обязательная работа + Bonus

Учебный комплект • 2026

Что требуется

Это одна работа на четыре последовательных шага: сначала форма и список, затем Material UI, затем DataGrid и в конце маршруты со статистикой. В готовом варианте все четыре шага собраны в одном приложении на React + TypeScript.

Поток данных такой:

`BudgetTracker` собирает `BudgetDraft` → `validateBudgetDraft` проверяет его → `App` вызывает `toBudgetItem` и хранит `BudgetItem[]` → `BudgetItems` показывает записи → `Statistics` считает итоговые значения из того же массива.

Что получится

В браузере будут три маршрута. На Budget пользователь добавляет и удаляет записи, а на Statistics видит общие доходы, расходы и saldo. Таблица поддерживает сортировку, фильтрацию и пагинацию.

Подготовка на macOS

Установите Node.js 20.19+ (или 22.12+) и проверьте версии командами `node --version` и `npm --version`. Работа не требует API-ключей или внешней базы данных.

Создание с нуля

КОД · BASH

```
npm create vite@latest budget-tracker -- --template react-ts
cd budget-tracker
npm install @mui/material @mui/x-data-grid @mui/x-date-pickers @emotion/react @emotion/styled
dayjs react-router-dom
npm install -D vitest jsdom @testing-library/react @testing-library/user-event @testing-
library/jest-dom
```

В учебном комплекте готовый проект уже находится в `projects/react-web/10-budget-tracker`.

Структура файлов

КОД · TEXT

```
src/
  components/BudgetTracker.tsx  # форма и месячный лимит
  components/BudgetItems.tsx    # DataGrid и удаление
  components/Home.tsx           # приветственный экран
  components/Statistics.tsx     # карточки итогов
  utils/budget.ts               # правила предметной области
  types.ts                      # типы TypeScript
  App.tsx                       # состояние и маршруты
```

Пошаговая реализация

Почему расходы отрицательные

Пользователь вводит обычное положительное число, например `12`. Тип операции определяет знак в одном месте — функции `toBudgetItem`: для `Expense` используется `-Math.abs(amount)`, для `Income` — `Math.abs(amount)`. Поэтому в остальных компонентах можно безопасно суммировать значения и не дублировать правило знака.

Что использовано и зачем

Инструмент/функция	Зачем
`useState`	хранить поля формы и список записей
props	передать записи и обработчики между компонентами
`validateBudgetDraft`	отделить правила проверки от JSX
`Math.abs`, `Number.isFinite`	нормализовать сумму и отсечь `NaN`/отрицательные значения
`dayjs` + `DatePicker`	ввод и строгая проверка даты в формате `YYYY-MM-DD`
MUI `TextField`, `NativeSelect`, `Button`, `Alert`, `AppBar`	единый интерфейс и сообщения об ошибках
MUI-X `DataGrid`	сортировка, фильтрация, пагинация и удаление строк
`BrowserRouter`, `Routes`, `Route`, `NavLink`	маршруты Home/Budget/Statistics
`calculateStatistics`	единый расчёт доходов, расходов и saldo
Vitest + Testing Library	проверка доменных правил, формы и сохранения состояния при переходе

Полные исходники

Ниже приведены все файлы, которые нужно разбирать на занятии. Путь указан относительно корня проекта.

src/types.ts

КОД · TS

```ts
export type BudgetType = 'Income' | 'Expense'

export type BudgetDraft = {
  description: string
  amount: string
  type: BudgetType
  date: string
}

export type BudgetItem = {
  id: string
  description: string
  amount: number
  type: BudgetType
  date: string
}

export type BudgetDraftErrors = Partial<Record<keyof BudgetDraft, string>>

export type BudgetStatistics = {
  income: number
  expenses: number
  balance: number
}
```

`BudgetDraft` хранит введённую сумму строкой: так форма может отличить пустое поле от нуля. В `BudgetItem` сумма уже число и готова к расчётам.

src/utils/budget.ts

КОД · TS

```ts
import dayjs from 'dayjs'
import customParseFormat from 'dayjs/plugin/customParseFormat'
import type {
  BudgetDraft,
  BudgetDraftErrors,
  BudgetItem,
  BudgetStatistics,
} from '../types'

dayjs.extend(customParseFormat)

const ISO_DATE_FORMAT = 'YYYY-MM-DD'

export function validateBudgetDraft(draft: BudgetDraft): BudgetDraftErrors {
  const errors: BudgetDraftErrors = {}
  const amount = Number(draft.amount)

  if (!draft.description.trim()) {
    errors.description = 'Description is required.'
  }

  if (!draft.amount.trim() || !Number.isFinite(amount) || amount < 0) {
    errors.amount = 'Amount must be a non-negative number.'
  }

  if (!draft.date || !dayjs(draft.date, ISO_DATE_FORMAT, true).isValid()) {
    errors.date = 'Date is required and must be valid.'
  }
```

```
  if (draft.type !== 'Income' && draft.type !== 'Expense') {
    errors.type = 'Type is required.'
  }

  return errors
}

export function toBudgetItem(draft: BudgetDraft, id: string): BudgetItem {
  const amount = Math.abs(Number(draft.amount))

  return {
    id,
    description: draft.description.trim(),
    amount: draft.type === 'Expense' ? -amount : amount,
    type: draft.type,
    date: draft.date,
  }
}

export function calculateStatistics(items: BudgetItem[]): BudgetStatistics {
  const income = items
    .filter((item) => item.type === 'Income')
    .reduce((total, item) => total + Math.abs(item.amount), 0)
  const expenses = items
    .filter((item) => item.type === 'Expense')
    .reduce((total, item) => total + Math.abs(item.amount), 0)

  return {
    income,
    expenses,
    balance: income - expenses,
  }
}

export function isCurrentMonth(date: string, today = dayjs().format(ISO_DATE_FORMAT)): boolean {
  return date.slice(0, 7) === today.slice(0, 7)
}
```

Ключевая идея — преобразовать данные один раз на границе формы, а не исправлять знак в каждой строке таблицы.

src/components/BudgetTracker.tsx

КОД · TSX

```
import { useState, type FormEvent } from 'react'
import {
  Alert,
  Box,
  Button,
  FormControl,
  FormHelperText,
  InputLabel,
  NativeSelect,
  Paper,
  Stack,
  TextField,
  Typography,
} from '@mui/material'
import { LocalizationProvider } from '@mui/x-date-pickers/LocalizationProvider'
import { DatePicker } from '@mui/x-date-pickers/DatePicker'
import { AdapterDayjs } from '@mui/x-date-pickers/AdapterDayjs'
import dayjs from 'dayjs'
import type { BudgetDraft, BudgetDraftErrors, BudgetItem, BudgetType } from '../types'
import { calculateStatistics, isCurrentMonth, validateBudgetDraft } from '../utils/budget'
import BudgetItems from './BudgetItems'
```

```
type BudgetTrackerProps = {
  items: BudgetItem[]
  monthlyLimit: number
  onAdd: (draft: BudgetDraft) => void
  onDelete?: (id: string) => void
}

const today = () => dayjs().format('YYYY-MM-DD')

const emptyDraft = (): BudgetDraft => ({
  description: '',
  amount: '',
  type: 'Income',
  date: today(),
})

export default function BudgetTracker({ items, monthlyLimit, onAdd, onDelete }:
BudgetTrackerProps) {
  const [draft, setDraft] = useState<BudgetDraft>(emptyDraft)
  const [errors, setErrors] = useState<BudgetDraftErrors>({})

  const monthlyExpenses = items
    .filter((item) => item.type === 'Expense' && isCurrentMonth(item.date))
    .reduce((total, item) => total + Math.abs(item.amount), 0)
  const statistics = calculateStatistics(items)

  function updateDraft(field: keyof BudgetDraft, value: string) {
    setDraft((current) => ({ ...current, [field]: value }))
    setErrors((current) => ({ ...current, [field]: undefined }))
  }

  function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()
    const nextErrors = validateBudgetDraft(draft)
    setErrors(nextErrors)
    if (Object.keys(nextErrors).length > 0) return

    onAdd(draft)
    setDraft(emptyDraft())
    setErrors({})
  }

  return (
    <Stack spacing={3}>
      <Paper className="page-heading" elevation={0}>
        <Typography className="eyebrow" component="p">
          Step 1 · Step 2 · Step 3
        </Typography>
        <Typography component="h1" variant="h4">Budget Tracker</Typography>
        <Typography color="text.secondary">
          Record income and expenses. Expenses are stored as negative values automatically.
        </Typography>
      </Paper>

      <Paper component="form" elevation={1} onSubmit={handleSubmit} sx={{ p: { xs: 2, md: 3 } }}>
        <Stack spacing={2}>
          {errors.description || errors.amount || errors.date ? (
            <Alert role="alert" severity="error">
              {[errors.description, errors.amount, errors.date].filter(Boolean).join(' ')}
            </Alert>
          ) : null}
          <Stack direction={{ md: 'row', xs: 'column' }} spacing={2}>
            <TextField
              error={Boolean(errors.description)}
              fullWidth
              helperText={errors.description}
```

```
              id="description"
              label="Description"
              onChange={(event) => updateDraft('description', event.target.value)}
              value={draft.description}
            />
            <TextField
              error={Boolean(errors.amount)}
              fullWidth
              helperText={errors.amount}
              id="amount"
              inputProps={{ inputMode: 'decimal', min: 0, step: '0.01' }}
              label="Amount"
              onChange={(event) => updateDraft('amount', event.target.value)}
              type="text"
              value={draft.amount}
            />
          </Stack>
          <Stack direction={{ md: 'row', xs: 'column' }} spacing={2}>
            <FormControl fullWidth error={Boolean(errors.type)}>
              <InputLabel htmlFor="type">Type</InputLabel>
              <NativeSelect
                id="type"
                inputProps={{ 'aria-label': 'Type' }}
                onChange={(event) => updateDraft('type', event.target.value as BudgetType)}
                value={draft.type}
              >
                <option value="Income">Income</option>
                <option value="Expense">Expense</option>
              </NativeSelect>
              <FormHelperText>{errors.type}</FormHelperText>
            </FormControl>
            <LocalizationProvider dateAdapter={AdapterDayjs}>
              <DatePicker
                format="YYYY-MM-DD"
                label="Date"
                onChange={(value) => updateDraft('date', value?.isValid() ? value.format('YYYY-MM-
DD') : '')}
                slotProps={{
                  textField: {
                    error: Boolean(errors.date),
                    fullWidth: true,
                    helperText: errors.date,
                  },
                }}
                value={draft.date ? dayjs(draft.date) : null}
              />
            </LocalizationProvider>
          </Stack>
          <Button size="large" type="submit" variant="contained">Add</Button>
        </Stack>
      </Paper>

      <Stack direction={{ md: 'row', xs: 'column' }} spacing={2}>
        <Paper className="summary-card" elevation={0}>
          <Typography color="text.secondary" variant="body2">Current balance</Typography>
          <Typography color={statistics.balance >= 0 ? 'success.main' : 'error.main'}
variant="h5">€{statistics.balance.toFixed(2)}</Typography>
        </Paper>
        <Paper className="summary-card" elevation={0}>
          <Typography color="text.secondary" variant="body2">Monthly expenses</Typography>
          <Typography variant="h5">€{monthlyExpenses.toFixed(2)}</Typography>
        </Paper>
      </Stack>

      {monthlyExpenses > monthlyLimit ? <Alert severity="warning">Monthly limit of
€{monthlyLimit.toFixed(2)} exceeded.</Alert> : null}
      <BudgetItems items={items} onDelete={onDelete ?? (() => undefined)} />
```

```
      <Box />
    </Stack>
  )
}
```

`BudgetTracker` владеет только временным состоянием формы. Массив записей остаётся в `App`, поэтому роутер не уничтожает данные при навигации.

src/components/BudgetItems.tsx

КОД · TSX

```tsx
import { Box, Button, Chip, Stack, Typography } from '@mui/material'
import { DataGrid, type GridColDef, type GridRowParams } from '@mui/x-data-grid'
import type { BudgetItem } from '../types'

type BudgetItemsProps = { items: BudgetItem[]; onDelete: (id: string) => void }

function formatAmount(amount: number) {
  return `${amount < 0 ? '-' : ''}€${Math.abs(amount).toFixed(2)}`
}

export default function BudgetItems({ items, onDelete }: BudgetItemsProps) {
  const columns: GridColDef<BudgetItem>[] = [
    { field: 'description', headerName: 'Description', flex: 1.4, minWidth: 170 },
    {
      field: 'amount',
      headerName: 'Amount',
      flex: 0.8,
      minWidth: 120,
      valueFormatter: (value) => formatAmount(Number(value)),
      cellClassName: (params) => (Number(params.value) < 0 ? 'expense-cell' : 'income-cell'),
    },
    { field: 'date', headerName: 'Date', flex: 0.8, minWidth: 130 },
    {
      field: 'type',
      headerName: 'Type',
      flex: 0.8,
      minWidth: 120,
      renderCell: (params) => (
        <Chip
          color={params.row.type === 'Income' ? 'success' : 'error'}
          label={params.row.type}
          size="small"
          variant="outlined"
        />
      ),
    },
    {
      field: 'actions',
      headerName: 'Actions',
      sortable: false,
      filterable: false,
      flex: 0.7,
      minWidth: 115,
      renderCell: (params) => (
        <Button
          aria-label={`Delete ${params.row.description}`}
          color="error"
          onClick={() => onDelete(params.row.id)}
          size="small"
        >
          Delete
        </Button>
      ),
    },
```

```
  ]
  const grouped = items.reduce(
    (result, item) => {
      result[item.type] += 1
      return result
    },
    { Income: 0, Expense: 0 },
  )

  return (
    <section aria-labelledby="budget-items-title">
      <Stack alignItems="center" direction="row" justifyContent="space-between" mb={1.5}>
        <Typography component="h2" id="budget-items-title" variant="h5">Budget items</Typography>
        <Stack direction="row" spacing={1}>
          <Chip color="success" label={`Income: ${grouped.Income}`} size="small" />
          <Chip color="error" label={`Expense: ${grouped.Expense}`} size="small" />
        </Stack>
      </Stack>
      {items.length === 0 ? (
        <Box className="empty-state">
          <Typography color="text.secondary">No budget items yet.</Typography>
        </Box>
      ) : (
        <DataGrid
          autoHeight
          columns={columns}
          disableRowSelectionOnClick
          getRowClassName={(params: GridRowParams<BudgetItem>) =>
            params.row.type === 'Income' ? 'income-row' : 'expense-row'
          }
          getRowId={(row) => row.id}
          initialState={{ pagination: { paginationModel: { page: 0, pageSize: 5 } } }}
          pageSizeOptions={[5, 10, 25]}
          rows={items}
          sx={{
            border: 0,
            '& .income-row': { backgroundColor: 'rgba(46, 125, 50, 0.04)' },
            '& .expense-row': { backgroundColor: 'rgba(211, 47, 47, 0.04)' },
            '& .income-cell': { color: 'success.main', fontWeight: 700 },
            '& .expense-cell': { color: 'error.main', fontWeight: 700 },
          }}
        />
      )}
    </section>
  )
}
```

`DataGrid` даёт сортировку, фильтрацию и пагинацию из коробки. Цвет строки и суммы определяется `type`, а счётчики сверху дают простую учебную группировку по типу без платного DataGridPro.

src/components/Statistics.tsx

КОД · TSX

```
import { Card, CardContent, Grid, Stack, Typography } from '@mui/material'
import type { BudgetItem } from '../types'
import { calculateStatistics } from '../utils/budget'

type StatisticsProps = { items: BudgetItem[] }

export default function Statistics({ items }: StatisticsProps) {
  const statistics = calculateStatistics(items)
  const cards = [
    { label: 'Total income', value: statistics.income, color: 'success.main' },
    { label: 'Total expenses', value: statistics.expenses, color: 'error.main' },
```

```
    { label: 'Saldo', value: statistics.balance, color: statistics.balance >= 0 ? 'success.main' :
'error.main' },
  ]

  return (
    <Stack spacing={3}>
      <Typography component="h1" variant="h4">Statistics</Typography>
      <Grid container spacing={2}>
        {cards.map((card) => (
          <Grid key={card.label} size={{ md: 4, xs: 12 }}>
            <Card elevation={1}>
              <CardContent>
                <Typography color="text.secondary" gutterBottom>{card.label}</Typography>
                <Typography color={card.color} variant="h4">
                  {card.label === 'Saldo' && card.value >= 0 ? '+' : ''}€{card.value.toFixed(2)}
                </Typography>
              </CardContent>
            </Card>
          </Grid>
        ))}
      </Grid>
    </Stack>
  )
}
```

src/components/Home.tsx

КОД · TSX

```
import { Box, Paper, Typography } from '@mui/material'

export default function Home() {
  return (
    <Paper className="hero-card" elevation={0}>
      <Box>
        <Typography className="eyebrow" component="p">Personal finance</Typography>
        <Typography component="h1" variant="h3" gutterBottom>Welcome to Budget
Tracker</Typography>
        <Typography color="text.secondary" sx={{ maxWidth: 620 }}>
          Add income and expenses, see the current balance, and use Statistics to understand where
your money goes.
        </Typography>
      </Box>
    </Paper>
  )
}
```

src/App.tsx

КОД · TSX

```
import { useState } from 'react'
import { AppBar, Box, Button, Container, Toolbar, Typography } from '@mui/material'
import { BrowserRouter, Link as RouterLink, NavLink, Route, Routes } from 'react-router-dom'
import type { BudgetDraft, BudgetItem } from './types'
import { toBudgetItem } from './utils/budget'
import Home from './components/Home'
import BudgetTracker from './components/BudgetTracker'
import Statistics from './components/Statistics'

function createId() {
  if (typeof globalThis.crypto?.randomUUID === 'function') return globalThis.crypto.randomUUID()
  return `${Date.now()}-${Math.random().toString(16).slice(2)}`
}
```

```
function Navigation() {
  return (
    <AppBar position="sticky"><Toolbar>
      <Typography component={RouterLink} sx={{ color: 'inherit', flexGrow: 1, textDecoration:
'none' }} to="/" variant="h6">Budget Tracker</Typography>
      <Button color="inherit" component={NavLink} to="/">Home</Button>
      <Button color="inherit" component={NavLink} to="/budget">Budget</Button>
      <Button color="inherit" component={NavLink} to="/statistics">Statistics</Button>
    </Toolbar></AppBar>
  )
}

export default function App() {
  const [items, setItems] = useState<BudgetItem[]>([])
  const monthlyLimit = 1000
  function addItem(draft: BudgetDraft) { setItems((current) => [...current, toBudgetItem(draft,
createId())]) }
  function deleteItem(id: string) { setItems((current) => current.filter((item) => item.id !==
id)) }

  return (
    <BrowserRouter><Navigation /><Box component="main" sx={{ py: { xs: 3, md: 5 } }}><Container
maxWidth="lg">
      <Routes>
        <Route element={<Home />} path="/" />
        <Route element={<BudgetTracker items={items} monthlyLimit={monthlyLimit} onAdd={addItem}
onDelete={deleteItem} />} path="/budget" />
        <Route element={<Statistics items={items} />} path="/statistics" />
        <Route element={<Home />} path="*" />
      </Routes>
    </Container></Box></BrowserRouter>
  )
}
```

Состояние находится выше маршрутов. Это и есть требование шага 4: при переходе с Budget на Statistics новый экран получает тот же `items`.

Точные листинги файлов

В этом приложении файлы приведены без сокращений и форматирования «в одну строку», чтобы их можно было копировать и разбирать построчно.

src/components/BudgetTracker.tsx — полный файл

КОД · TSX

```
import { useState, type FormEvent } from 'react'
import {
  Alert,
  Box,
  Button,
  FormControl,
  FormHelperText,
  InputLabel,
  NativeSelect,
  Paper,
  Stack,
  TextField,
  Typography,
} from '@mui/material'
import { LocalizationProvider } from '@mui/x-date-pickers/LocalizationProvider'
import { DatePicker } from '@mui/x-date-pickers/DatePicker'
import { AdapterDayjs } from '@mui/x-date-pickers/AdapterDayjs'
```

```
import dayjs from 'dayjs'
import type { BudgetDraft, BudgetDraftErrors, BudgetItem, BudgetType } from '../types'
import { calculateStatistics, isCurrentMonth, validateBudgetDraft } from '../utils/budget'
import BudgetItems from './BudgetItems'

type BudgetTrackerProps = {
  items: BudgetItem[]
  monthlyLimit: number
  onAdd: (draft: BudgetDraft) => void
  onDelete?: (id: string) => void
}

const today = () => dayjs().format('YYYY-MM-DD')

const emptyDraft = (): BudgetDraft => ({
  description: '',
  amount: '',
  type: 'Income',
  date: today(),
})

export default function BudgetTracker({ items, monthlyLimit, onAdd, onDelete }:
BudgetTrackerProps) {
  const [draft, setDraft] = useState<BudgetDraft>(emptyDraft)
  const [errors, setErrors] = useState<BudgetDraftErrors>({})

  const monthlyExpenses = items
    .filter((item) => item.type === 'Expense' && isCurrentMonth(item.date))
    .reduce((total, item) => total + Math.abs(item.amount), 0)
  const statistics = calculateStatistics(items)

  function updateDraft(field: keyof BudgetDraft, value: string) {
    setDraft((current) => ({ ...current, [field]: value }))
    setErrors((current) => ({ ...current, [field]: undefined }))
  }

  function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()
    const nextErrors = validateBudgetDraft(draft)
    setErrors(nextErrors)
    if (Object.keys(nextErrors).length > 0) return

    onAdd(draft)
    setDraft(emptyDraft())
    setErrors({})
  }

  return (
    <Stack spacing={3}>
      <Paper className="page-heading" elevation={0}>
        <Typography className="eyebrow" component="p">
          Step 1 · Step 2 · Step 3
        </Typography>
        <Typography component="h1" variant="h4">
          Budget Tracker
        </Typography>
        <Typography color="text.secondary">
          Record income and expenses. Expenses are stored as negative values automatically.
        </Typography>
      </Paper>

      <Paper component="form" elevation={1} onSubmit={handleSubmit} sx={{ p: { xs: 2, md: 3 } }}>
        <Stack spacing={2}>
          {errors.description || errors.amount || errors.date ? (
            <Alert role="alert" severity="error">
              {[errors.description, errors.amount, errors.date].filter(Boolean).join(' ')}
            </Alert>
```

```
          ) : null}
          <Stack direction={{ md: 'row', xs: 'column' }} spacing={2}>
            <TextField
              error={Boolean(errors.description)}
              fullWidth
              helperText={errors.description}
              id="description"
              label="Description"
              onChange={(event) => updateDraft('description', event.target.value)}
              value={draft.description}
            />
            <TextField
              error={Boolean(errors.amount)}
              fullWidth
              helperText={errors.amount}
              id="amount"
              inputProps={{ inputMode: 'decimal', min: 0, step: '0.01' }}
              label="Amount"
              onChange={(event) => updateDraft('amount', event.target.value)}
              type="text"
              value={draft.amount}
            />
          </Stack>
          <Stack direction={{ md: 'row', xs: 'column' }} spacing={2}>
            <FormControl fullWidth error={Boolean(errors.type)}>
              <InputLabel htmlFor="type">Type</InputLabel>
              <NativeSelect
                id="type"
                inputProps={{ 'aria-label': 'Type' }}
                onChange={(event) => updateDraft('type', event.target.value as BudgetType)}
                value={draft.type}
              >
                <option value="Income">Income</option>
                <option value="Expense">Expense</option>
              </NativeSelect>
              <FormHelperText>{errors.type}</FormHelperText>
            </FormControl>
            <LocalizationProvider dateAdapter={AdapterDayjs}>
              <DatePicker
                format="YYYY-MM-DD"
                label="Date"
                onChange={(value) => updateDraft('date', value?.isValid() ? value.format('YYYY-MM-
DD') : '')}
                slotProps={{
                  textField: {
                    error: Boolean(errors.date),
                    fullWidth: true,
                    helperText: errors.date,
                  },
                }}
                value={draft.date ? dayjs(draft.date) : null}
              />
            </LocalizationProvider>
          </Stack>
          <Button size="large" type="submit" variant="contained">
            Add
          </Button>
        </Stack>
      </Paper>

      <Stack direction={{ md: 'row', xs: 'column' }} spacing={2}>
        <Paper className="summary-card" elevation={0}>
          <Typography color="text.secondary" variant="body2">Current balance</Typography>
          <Typography color={statistics.balance >= 0 ? 'success.main' : 'error.main'}
variant="h5">
            €{statistics.balance.toFixed(2)}
          </Typography>
```

```
        </Paper>
        <Paper className="summary-card" elevation={0}>
          <Typography color="text.secondary" variant="body2">Monthly expenses</Typography>
          <Typography variant="h5">€{monthlyExpenses.toFixed(2)}</Typography>
        </Paper>
      </Stack>

      {monthlyExpenses > monthlyLimit ? (
        <Alert severity="warning">
          Monthly limit of €{monthlyLimit.toFixed(2)} exceeded.
        </Alert>
      ) : null}

      <BudgetItems items={items} onDelete={onDelete ?? (() => undefined)} />
      <Box />
    </Stack>
  )
}
```

src/components/BudgetItems.tsx — полный файл

КОД · TSX

```
import { Box, Button, Chip, Stack, Typography } from '@mui/material'
import { DataGrid, type GridColDef, type GridRowParams } from '@mui/x-data-grid'
import type { BudgetItem } from '../types'

type BudgetItemsProps = {
  items: BudgetItem[]
  onDelete: (id: string) => void
}

function formatAmount(amount: number) {
  return `${amount < 0 ? '-' : ''}€${Math.abs(amount).toFixed(2)}`
}

export default function BudgetItems({ items, onDelete }: BudgetItemsProps) {
  const columns: GridColDef<BudgetItem>[] = [
    { field: 'description', headerName: 'Description', flex: 1.4, minWidth: 170 },
    {
      field: 'amount',
      headerName: 'Amount',
      flex: 0.8,
      minWidth: 120,
      valueFormatter: (value) => formatAmount(Number(value)),
      cellClassName: (params) => (Number(params.value) < 0 ? 'expense-cell' : 'income-cell'),
    },
    { field: 'date', headerName: 'Date', flex: 0.8, minWidth: 130 },
    {
      field: 'type',
      headerName: 'Type',
      flex: 0.8,
      minWidth: 120,
      renderCell: (params) => (
        <Chip
          color={params.row.type === 'Income' ? 'success' : 'error'}
          label={params.row.type}
          size="small"
          variant="outlined"
        />
      ),
    },
    {
      field: 'actions',
      headerName: 'Actions',
      sortable: false,
```

```
      filterable: false,
      flex: 0.7,
      minWidth: 115,
      renderCell: (params) => (
        <Button
          aria-label={`Delete ${params.row.description}`}
          color="error"
          onClick={() => onDelete(params.row.id)}
          size="small"
        >
          Delete
        </Button>
      ),
    },
  ]

  const grouped = items.reduce(
    (result, item) => {
      result[item.type] += 1
      return result
    },
    { Income: 0, Expense: 0 },
  )

  return (
    <section aria-labelledby="budget-items-title">
      <Stack alignItems="center" direction="row" justifyContent="space-between" mb={1.5}>
        <Typography component="h2" id="budget-items-title" variant="h5">
          Budget items
        </Typography>
        <Stack direction="row" spacing={1}>
          <Chip color="success" label={`Income: ${grouped.Income}`} size="small" />
          <Chip color="error" label={`Expense: ${grouped.Expense}`} size="small" />
        </Stack>
      </Stack>
      {items.length === 0 ? (
        <Box className="empty-state">
          <Typography color="text.secondary">No budget items yet.</Typography>
        </Box>
      ) : (
        <Box sx={{ width: '100%' }}>
          <DataGrid
            autoHeight
            columns={columns}
            disableRowSelectionOnClick
            getRowClassName={(params: GridRowParams<BudgetItem>) =>
              params.row.type === 'Income' ? 'income-row' : 'expense-row'
            }
            getRowId={(row) => row.id}
            initialState={{ pagination: { paginationModel: { page: 0, pageSize: 5 } } }}
            pageSizeOptions={[5, 10, 25]}
            rows={items}
            sx={{
              border: 0,
              '& .income-row': { backgroundColor: 'rgba(46, 125, 50, 0.04)' },
              '& .expense-row': { backgroundColor: 'rgba(211, 47, 47, 0.04)' },
              '& .income-cell': { color: 'success.main', fontWeight: 700 },
              '& .expense-cell': { color: 'error.main', fontWeight: 700 },
            }}
          />
        </Box>
      )}
    </section>
  )
}
```

src/components/Home.tsx — полный файл

КОД · TSX

```tsx
import { Box, Paper, Typography } from '@mui/material'

export default function Home() {
  return (
    <Paper className="hero-card" elevation={0}>
      <Box>
        <Typography className="eyebrow" component="p">
          Personal finance
        </Typography>
        <Typography component="h1" variant="h3" gutterBottom>
          Welcome to Budget Tracker
        </Typography>
        <Typography color="text.secondary" sx={{ maxWidth: 620 }}>
          Add income and expenses, see the current balance, and use Statistics
          to understand where your money goes.
        </Typography>
      </Box>
    </Paper>
  )
}
```

src/App.tsx — полный файл

КОД · TSX

```tsx
import { useState } from 'react'
import { AppBar, Box, Button, Container, Toolbar, Typography } from '@mui/material'
import { BrowserRouter, Link as RouterLink, NavLink, Route, Routes } from 'react-router-dom'
import type { BudgetDraft, BudgetItem } from './types'
import { toBudgetItem } from './utils/budget'
import Home from './components/Home'
import BudgetTracker from './components/BudgetTracker'
import Statistics from './components/Statistics'

function createId() {
  if (typeof globalThis.crypto?.randomUUID === 'function') return globalThis.crypto.randomUUID()
  return `${Date.now()}-${Math.random().toString(16).slice(2)}`
}

function Navigation() {
  return (
    <AppBar position="sticky">
      <Toolbar>
        <Typography component={RouterLink} sx={{ color: 'inherit', flexGrow: 1, textDecoration:
'none' }} to="/" variant="h6">
          Budget Tracker
        </Typography>
        <Button color="inherit" component={NavLink} to="/">Home</Button>
        <Button color="inherit" component={NavLink} to="/budget">Budget</Button>
        <Button color="inherit" component={NavLink} to="/statistics">Statistics</Button>
      </Toolbar>
    </AppBar>
  )
}

export default function App() {
  const [items, setItems] = useState<BudgetItem[]>([])
  const monthlyLimit = 1000

  function addItem(draft: BudgetDraft) {
    setItems((current) => [...current, toBudgetItem(draft, createId())])
  }
```

```
  function deleteItem(id: string) {
    setItems((current) => current.filter((item) => item.id !== id))
  }

  return (
    <BrowserRouter>
      <Navigation />
      <Box component="main" sx={{ py: { xs: 3, md: 5 } }}>
        <Container maxWidth="lg">
          <Routes>
            <Route element={<Home />} path="/" />
            <Route
              element={<BudgetTracker items={items} monthlyLimit={monthlyLimit} onAdd={addItem}
onDelete={deleteItem} />}
              path="/budget"
            />
            <Route element={<Statistics items={items} />} path="/statistics" />
            <Route element={<Home />} path="*" />
          </Routes>
        </Container>
      </Box>
    </BrowserRouter>
  )
}
```

src/main.tsx — полный файл

КОД · TSX

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App'
import './index.css'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/index.css — полный файл

КОД · CSS

```
:root {
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI",
sans-serif;
  color: #172033;
  background: #f5f7fb;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* { box-sizing: border-box; }
body { margin: 0; min-width: 320px; }
.hero-card, .page-heading { padding: clamp(1.5rem, 4vw, 3rem); }
.hero-card { min-height: 320px; display: flex; align-items: center; background: linear-
gradient(135deg, #e9f4ff 0%, #fff 70%); }
.page-heading { background: linear-gradient(135deg, #eef7ff 0%, #fff 75%); }
.eyebrow { color: #1976d2; font-size: .78rem; font-weight: 800; letter-spacing: .1em; text-
transform: uppercase; }
.summary-card { flex: 1; padding: 1.25rem 1.5rem; background: #fff; }
.empty-state { border: 1px dashed #b7c4d8; border-radius: 12px; padding: 2.5rem; text-align:
center; background: rgba(255,255,255,.7); }
```

src/main.tsx

КОД · TSX

```tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App'
import './index.css'

createRoot(document.getElementById('root')!).render(
  <StrictMode><App /></StrictMode>,
)
```

src/index.css

КОД · CSS

```css
:root {
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI",
sans-serif;
  color: #172033;
  background: #f5f7fb;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}
* { box-sizing: border-box; }
body { margin: 0; min-width: 320px; }
.hero-card, .page-heading { padding: clamp(1.5rem, 4vw, 3rem); }
.hero-card { min-height: 320px; display: flex; align-items: center; background: linear-
gradient(135deg, #e9f4ff 0%, #fff 70%); }
.page-heading { background: linear-gradient(135deg, #eef7ff 0%, #fff 75%); }
.eyebrow { color: #1976d2; font-size: .78rem; font-weight: 800; letter-spacing: .1em; text-
transform: uppercase; }
.summary-card { flex: 1; padding: 1.25rem 1.5rem; background: #fff; }
.empty-state { border: 1px dashed #b7c4d8; border-radius: 12px; padding: 2.5rem; text-align:
center; background: rgba(255,255,255,.7); }
```

Запуск и тесты

КОД · BASH

```bash
cd front-end-sof004as3ae-3008-budget-tracker-pe010101
npm install --include=optional
npm run test
npm run build
npm run dev
```

Для Vite 8 нужен Node.js `20.19+` или `22.12+`. Если npm на Windows пропустил native optional binding Rolldown, повторите установку `npm install --include=optional`; для уже созданного проекта допустима команда `npm install --no-save @rolldown/binding-win32-x64-msvc@1.2.6`.

Ручная проверка

1. Открыть `/` — видна приветственная страница.
2. Перейти на `/budget`.
3. Нажать Add с пустыми полями — появляется ошибка, запись не создаётся.
4. Ввести `Salary`, `450`, `Income`, дату и нажать Add — появляется зелёная строка.
5. Ввести `Lunch`, `12`, `Expense` — сумма отображается как `-€12.00` и выделяется красным.

6. Нажать Delete — строка удаляется.
7. Добавить запись, перейти на Statistics — доход, расходы и saldo учитывают запись.
8. Добавить расходов больше `€1000` за текущий месяц — появляется предупреждение лимита.

Типичные ошибки

- забыть `LocalizationProvider` вокруг `DatePicker`;
- хранить расход положительным и затем отдельно вычитать его в нескольких компонентах;
- держать `items` внутри страницы Budget и терять их при смене маршрута;
- забыть `getRowId`, если id не называется `id`.

Вопросы и ответы

Почему `amount` в draft — строка? Чтобы отличить пустой ввод от числа `0` и показать правильную ошибку.

Где выполняется проверка даты? В `validateBudgetDraft`, где `dayjs` разбирает дату строго по `YYYY-MM-DD`.

Почему Statistics не получает отдельную копию списка? Ему передаётся тот же массив из `App`, поэтому данные синхронны.

Защита

Объясните устно: где находятся props, где state, где меняется знак Expense, зачем нужен DataGrid и почему состояние поднято в App.

Чек-лист сдачи

- ☐ `npm run test` проходит;
- ☐ `npm run build` проходит;
- ☐ пустая форма показывает ошибку;
- ☐ доход и расход имеют правильные знаки и цвета;
- ☐ Delete удаляет строку;
- ☐ переход на Statistics сохраняет запись.